

Machine Learning Project Report (Checkpoint-1)

Course: AIT 511 - Machine Learning

Team Name: Hidden Outliers

Shreeya H (IMT2022535)

Santhosh Kiran (IMT2023065)

Sriram Srikanth (IMT2023115)

26 October 2025

1 About the dataset

Transporting sensitive medical equipment comes with unique challenges:

- Ensuring fragile devices arrive safely.
- Handling urgent shipments for critical hospital needs.
- Managing cross-border deliveries and rural hospital locations.
- Coordinating installation and setup services when required.

The company whose data is in the dataset, specializes in delivering medical equipment sourced from multiple suppliers worldwide. The goal is to build a model that predicts the transport cost of each order, enabling better logistics planning and fair pricing.

We are given the dataset of hospital equipment transportation. The goal is to build a model that predicts the transport cost of each order, enabling better logistics planning and fair pricing.

The given problem is a regression problem whose input features consist of 19 features as displayed in the table below and the output label is the **Transport Cost**

You can find below the dataset link and the code github repository link:

- **Dataset Link**
- **Project Github Repository Link**



Column Name	Description
Hospital_Id	Unique identification number of the hospital
Supplier_Name	Name of the medical equipment supplier
Supplier_Reliability	Supplier's reputation score in the market (higher means more reliable)
Equipment_Height	Height of the equipment
Equipment_Width	Width of the equipment
Equipment_Weight	Weight of the equipment
Equipment_Type	The type of equipment (e.g., diagnostic device, surgical tool)
Equipment_Value	Monetary value of the equipment
Base_Transport_Fee	Base fee for transporting the equipment
CrossBorder_Shipping	Whether the delivery is international
Urgent_Shipping	Whether the delivery was in express/urgent mode
Installation_Service	Whether installation/setup service was included
Transport_Method	Mode of transport used (e.g., air, road, sea)
Fragile_Equipment	Whether the equipment is fragile
Hospital_Info	Additional information about the hospital
Rural_Hospital	Whether the hospital is located in a remote/rural area
Order_Placed_Date	Date when the order was placed
Delivery_Date	Date when the delivery occurred
Hospital_Location	Location of the hospital
Transport_Cost	Target variable — cost of transporting the order

Table 1: Columns of the Hospital Equipment Transport Dataset

2 Analysis of the Dataset

We analysed the training as well as test dataset using several methods:

- **Correlation Maps:** We analysed which features would be more likely to directly increase/decrease with respect to each other thereby creating easier patterns to recognise.

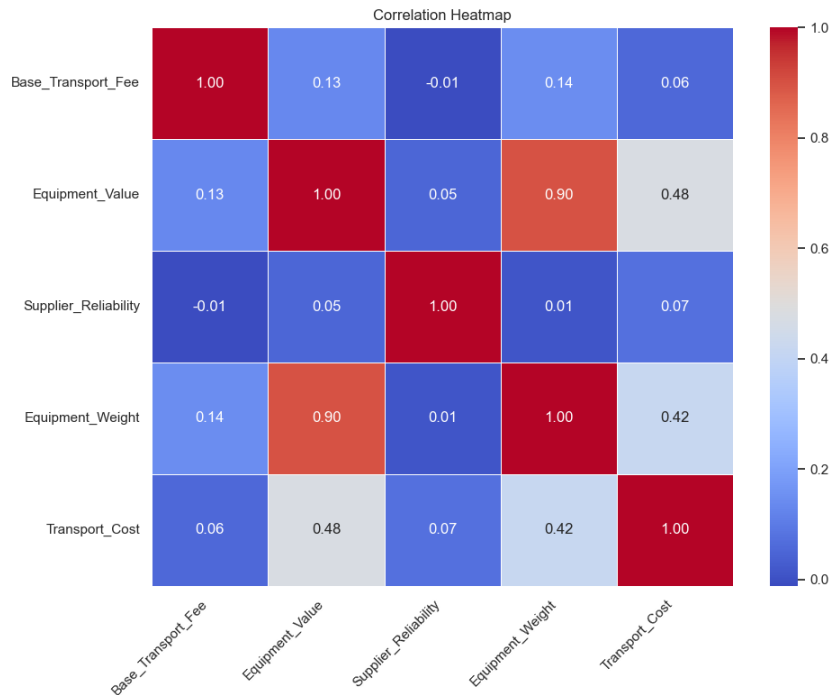


Figure 1: Correlation Heatmap

- **Histograms:** We used histograms to analyse distribution of data and the frequency at which each range of values of a given numeric column is occurring.
- **Boxplots:** Boxplots were used to examine each numerical feature for potential outliers, which appear as points beyond the plot's whiskers. The analysis revealed that key features like Transport_Cost, Base_Transport_Fee and Equipment_Value are heavily right skewed and many outliers.

3 Data Preprocessing

3.1 Removing the NaN and illogical values

Firstly we **removed any duplicate data points** in the training dataset and stored the Hospital Id column values of the test dataset so that it can be used for building our submission csv files.

Secondly, we **clean up the string data** in the dataset by converting all the string values to the string datatype, removing trail whitespaces and converting all of them to lowercase to standardise the strings. Similarly the method by which are **handling negative values** in Transport Cost and Delivery Days is by setting them to 0.

On examining the dataset we see the following profile of NaN values in the dataset:

Profile of NaN values in training data	
Hospital_Id	0
Supplier_Name	0
Supplier_Reliability	694
Equipment_Height	339
Equipment_Width	544
Equipment_Weight	549
Equipment_Type	710
Equipment_Value	0
Base_Transport_Fee	0
CrossBorder_Shipping	0
Urgent_Shipping	0
Installation_Service	0
Transport_Method	1298
Fragile_Equipment	0
Hospital_Info	0
Rural_Hospital	719
Order_Placed_Date	0
Delivery_Date	0
Hospital_Location	0
Transport_Cost	0
dtype:	int64

Figure 2: Profile of NaN values in training data

Profile of NaN values in test data	
Hospital_Id	0
Supplier_Name	0
Supplier_Reliability	55
Equipment_Height	23
Equipment_Width	56
Equipment_Weight	52
Equipment_Type	56
Equipment_Value	0
Base_Transport_Fee	0
CrossBorder_Shipping	0
Urgent_Shipping	0
Installation_Service	0
Transport_Method	123
Fragile_Equipment	0
Hospital_Info	0
Rural_Hospital	64
Order_Placed_Date	0
Delivery_Date	0
Hospital_Location	0
dtype:	int64

Figure 3: Profile of NaN values in test data

For removing the NaN values from the data we use sklearn's **SimpleImputer** which imputes the NaN values in the training dataset using median/mode/fixed value and then uses the same imputed values to impute the NaN in the test dataset.

We have used:

- **Mode:** For categorical columns - Equipment_Type, Transport_Method and Rural_Hospital
- **Median:** For continuous data columns - Supplier_Reliability, Equipment_Height, Equipment_Width, Equipment_Weight since it is not affected by outliers unlike mean.

3.2 Feature Engineering

We have added the following additional features as a part of feature engineering in order to enable the model to better understand the data.

a) **Date based features:** We have added such features to capture temporal patterns and service duration which help detect situations like weekend rushes, delay due to weather conditions, bussiness cycles.

- *Day of month:* The day of placing order and the day on which it was delivered
- *Month:* The month of placing order and the day on which it was delivered
- *Weekday:* The day of the week of placing order and the day on which it was delivered
- *Day of year:* The day of the year of placing order and the day on which it was delivered
- *Weekday:* Whether the day of placing order and the day on which it was delivered was a weekday or on the weekend
- *Delivery Days:* Number of days it took for the item to be delivered. For some entries this value is negative and hence we have capped those values at 0.

b) **Interaction based features:** These are features created to capture the combined effect of two or more features

- *Value_per_weight:* This feature captures the value density of an equipment enabling the model to differentiate between heavy,cheap equipment and light,expensive equipment
- *Fee_per_weight:* This feature calculates the transportation cost per unit weight and helps understand the underlying pricing structure for heavier items vs lightweight items.
- *Value_to_to_fee_ratio:* It calculates the shipping cost relative to the item's worth giving the model context on cost sensitivity for the equipment i.e. how much deos transport cost for that prize range equipment.
- *Delivery_Days_per_reliability:* This encorporates the risk factor of the supplier i.e. supplier reliability into the delivery dates thereby giving us a risk adjusted delivery timeline.
- *Value_per_day:* This can be seen as the cost of equipment transported per day. For example an high value of this would mean a very expensive item is being moved very quickly thereby enabling the model to capture features like premium transportation, urgency,etc.
- *Fragile_and_urgent:* It identifies the combined effect of premium handling and delivery of the equipment enabling the model to predict that this data point's cost is going to be relatively higher when compared to the ones where this value is 0/false.
- *CrossBorder_and_urgent:* It identifies the combined effect of cross border shipping and quick delivery. Similar to the above feature this also helps the model understand the chances of the cost of a row with this as true to be more than that with this value as false is high.

c) **Location based feature:** The Hospital_Location column has been replaced by a Hospital_State column thereby regularising the address of the Hospitals enabling better predictions and encoding of this column. The Hospital State is computed by taking the first string in the address before the gap or a comma. Ex: "Jasonfort, AR 42638" becomes "Jansonfort"

d) **Aggregation based features:**

- For this we have considered string features - Supplier_Name, Equipment_Type, Hospital_State and for each of these we have considered features which are correlated with these features.

- Then we do a group by on the string features and find aggregations of the correlated columns for these groups (by applying mean, standard deviation and maximum)

String Features	Related Features
Supplier_Name	Supplier_Reliability, Equipment_Value, Base_Transport_Fee, Delivery_Days
Equipment_Type	Equipment_Weight, Equipment_Value, Base_Transport_Fee
Hospital_State	Supplier_Reliability, Delivery_Days, Base_Transport_Fee, Equipment_Value

Table 2: Aggregated Features Mapping

3.3 Removing the outliers

We remove the outliers by:

- Computing lower and upper bounds of the numeric data columns based on quantile - 0.5% percentile value as lower and 99.5% percentile value as upper.
- Instead of dropping the rows which fall out of this range we impute these values with the help of the lower and upper values to prevent unnecessary data loss.

3.4 Balancing the skewed data

From the data analysis we observed that the target column and numeric columns are skewed. The target column was right skewed hence we applied $\log(1+x)$ onto that column which was sufficient enough to remove the skewness from the column data.

For other numeric columns applying $\log(1+x)$ or $\exp(x)$ was slightly effective but did not improve the model's accuracy hence I applied Power Transformer on the numeric columns.

We transform the data to look as close to the normal distribution by using sklearn's **Power Transformer** in order to remove outliers.

Also this helps the StandardScaler scale using valid mean and standard deviation which is reliable.

How PowerTransformer works ? It transforms a data x using a function of x, λ where the λ is chosen by first selecting a random value and then continually optimizing for the log-likelihood of the data wrt normal distribution.

$$y(\lambda) = \begin{cases} \frac{\lambda}{[(y+1)^\lambda - 1]} & \text{if } \lambda \neq 0, y \geq 0 \\ \log(y+1) & \text{if } \lambda = 0, y \geq 0 \\ \frac{2-\lambda}{[(-y+1)^{2-\lambda} - 1]} & \text{if } \lambda \neq 2, y < 0 \\ -\log(-y+1) & \text{if } \lambda = 2, y < 0 \end{cases}$$

3.5 Encoding the non-numeric data

We apply two types of encoding on the non-numeric data depending on the cardinality of the unique values it has:

1. **One-Hot Encoding:** For low cardinality columns i.e. - Equipment_Type, CrossBorder_Shipping, Urgent_Shipping, Installation_Service, Transport_Method, Fragile_Equipment, Hospital_Info, Rural_Hospital Here we take each column and convert it into a new binary column for each category in the column where 1/True indicates the data point's presence in the category and 0/False indicates its absence in the category.
2. **K-Fold Target Encoding:** For high-cardinality columns i.e. - Hospital_Id, Supplier_Name, Order_Placed_Date, Delivery_Date, Hospital_State
 - The idea of target encoding is to use the average of the target values of a particular category to replace it, but this would lead to overfitting.
 - Therefore, we apply KFold target encoding wherein we divide the training data into k independent chunks and for each chunk we compute the average target value of the other k-1 chunks and combine it with the global mean of the target value to encode that chunk.

$$\text{encoded_value} = \frac{\text{counts} \times \text{means} + \text{smoothing} \times \text{global_mean}}{\text{counts} + \text{smoothing}}$$

where:

- **counts** = number of occurrences of the category in k-1 chunks
 - **mean** = mean of target value for that category data points in k-1 chunks
 - **smoothing** = hyperparameter which helps avoid overfitting by handling rare categories (like weighted average)
 - **global_mean** = mean of the target column in all k chunks.
- For test data we cannot use kFold since the target value is not available. So we for any categories existing in the test data which is in training data we apply the formula for encoded_value same as above but considered the entire training dataset. For those categories which are newly observed in the test data we use the global mean of the training data.
 - On finetuning the hyperparameters - number of splits and smoothing we found k = 20 and smoothing = 40 to work the best for our dataset. This might be due to the fact that there are many unique categories in these columns (since Hospital_Id and Supplier_Name, etc.. may not have many repetitions of the same id or name)

3.6 Scaling the data to avoid large values

We scale the numeric and target encoded columns using the StandardScaler by sklearn which scales down the data using the following formula:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

where X is the original value, μ is the mean, and σ is the standard deviation.

4 Prediction attempts using various models

Libraries used

numpy, pandas, matplotlib, scikit-learn, xgboost, catboost, lightgbm

4.1 Ordinary Linear Regression

- It fits a straight-line relationship between numeric features (Supplier_Reliability, Equipment_Height/ Equipment_Width/ Equipment_Weight/ Equipment_Value, Base_Transport_Fee, etc.) and Transport_Cost.
- Captures straightforward, linear effects well: heavier or more valuable equipment generally increases cost.
- Its limitation is that it is sensitive to multicollinearity (e.g., Fee_per_weight vs Value_per_weight) and cannot fully capture complex interactions (e.g., urgent + fragile + rural deliveries).

It gives an RMSE of **44730.772** on the validation dataset and a MSE of **6159110982.712** on the test dataset.

RMSE: 44730.7720

Figure 4: RMSE of the predictions on validation dataset by Ordinary Linear Regression model

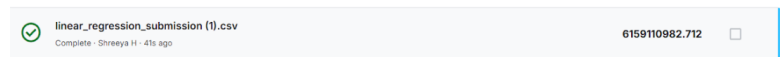


Figure 5: MSE of the predictions on test dataset by Ordinary Linear Regression model

4.2 Ridge Regression

- Handles the numeric features and engineered features like OLS, but applies L2 penalty to shrink coefficients without eliminating them..
- Keeps all interdependent features (Fee_per_weight, Value_per_weight, Delivery_Days) in the model, preserving predictive power.
- It is the best among linear models for this dataset because it reduces overfitting from correlated features while retaining their signal.
- But it is still limited in capturing extreme non-linear interactions beyond polynomial terms

Hyperparameters:

Hyperparameter	Meaning	Value
alpha	controls how much to regularise the linear regression	2.465

Table 3: Hyperparameters Description

It gives an RMSE of **44739.314** on the validation dataset and a MSE of **60591855.712** on the test dataset.

RMSE: 44739.3144

Figure 6: RMSE of the predictions on validation dataset by Ridge Regression model

ridge_submission (1).csv
Complete · Shreeya H · 18m ago
6057918555.426

Figure 7: MSE of the predictions on test dataset by Ridge Regression model

4.3 Lasso Regression

- Performs worse than Ridge because removing relevant correlated features reduces predictive accuracy.
- Feature selection property is not beneficial here since most features are informative.
- Predictions can be unstable when small changes occur in correlated columns.

Hyperparameters:

Hyperparameter	Meaning	Value
alpha	controls how much to regularise the linear regression	0.008

Table 4: Hyperparameters Description

It gives an RMSE of **45210.959** on the validation dataset and a MSE of **6084636990.011** on the test dataset.

RMSE: 45210.9596

Figure 8: RMSE of the predictions on validation dataset by Lasso Regression model

lasso_poly_randomsearch (1).csv
Complete · Shreeya H · 1h ago
6084636990.011

Figure 9: MSE of the predictions on test dataset by Lasso Regression model

4.4 ElasticNet Regression

- Combines L1 (LASSO) and L2 (Ridge) penalties to balance feature selection and coefficient shrinkage.
- Keeps correlated features partially, but the mixed penalty sometimes over-regularizes, limiting flexibility.
- Among linear models gives the best validation mse score and 2nd best test mse score.
- Underfits due to excessive shrinkage, losing predictive signal while discarding minor features.

Hyperparameters:

Hyperparameter	Meaning	Value
alpha	controls the total amount of L1 and L2 regularization applied to the model.	0.017
l1_ratio	lies between 0 and 1 that defines the mix of penalties, where 1 is pure L1 (Lasso) and 0 is pure L2 (Ridge)	0.95

Table 5: Hyperparameters Description

It gives an RMSE of **45550.297** on the validation dataset and a MSE of **6097539567.533** on the test dataset.

RMSE: 45550.2978

Figure 10: RMSE of the predictions on validation dataset by Elastic Net Regression model

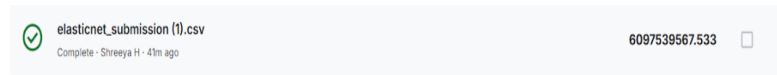


Figure 11: MSE of the predictions on test dataset by Elastic Net Regression model

4.5 Bayesian Regression

Bayesian regression like the above linear regression models tries its best to capture all the linear and polynomial relationships through MAP estimates, but it is unable to capture the non-linear complex relationships exhibited by this dataset. Hence it performs poorly on the test data.

Hyperparameter	Meaning	Value
max_iters	Number iterations the algorithm should run for	5000

Table 6: Hyperparameters Description

It gives an RMSE of **45551.300** on the validation dataset and a MSE of **6122869795.999** on the test dataset.

RMSE: 45551.3009

Figure 12: RMSE of the predictions on validation dataset by Bayesian Regression model



Figure 13: MSE of the predictions on test dataset by Bayesian Regression model

4.6 KNN-Regression Model

We see that KNN works moderately well on training data but not on test data despite a lot of fine tuning k. This might be due to the fact knn is a lazy algorithm which cannot learn which of the features

are important and treats all equally only using distance to regress the output. Hence it overfits to training data is unable to capture non-linear relationships.

Hyperparameters

Hyperparameter	Meaning	Value
n_neighbors	Number of neighbors of the data used to compute test data' target value	100

Table 7: Hyperparameters Description

It gives an RMSE of **46860.685** on the validation dataset and a MSE of **6158686669.825** on the test dataset.

RMSE: 46860.6853

Figure 14: RMSE of the predictions on validation dataset by KNN model



Figure 15: MSE of the predictions on test dataset by KNN model

4.7 RandomForest Regression

- It is a bagging based model which builds an ensemble of decision trees trained on different subsets of data and features.
- It captures nonlinear relationships and complex feature interactions effectively. However, deeper trees tend to memorize the training data, increasing the risk of overfitting.
- Low Training MSE — fits the training data very closely, especially with higher tree depth. Noticeable increase in Test MSE — overfitting becomes evident as depth increases, indicating poor generalization.
- Performs best when tree depth and number of estimators are carefully balanced to control variance.

Hyperparameters

Hyperparameter	Meaning	Value
n_estimators	total number of individual decision trees to build in the forest	500
max_depth	the maximum depth for any single tree, controlling its complexity	8
min_samples_split	requires a node to have at least that many samples before it is allowed to be split further	5
min_samples_leaf	ensures every final leaf node must have at least that many samples in it	1
max_features	considers only a random that much % of the total features when looking for the best split at each node	0.8

Table 8: Hyperparameters Description

It gives an RMSE of **38316.815** on the validation dataset and a MSE of **4353868078.73** on the test dataset.

RMSE: 38316.81503335057

Figure 16: RMSE of the predictions on validation dataset by Random Forest Regression model

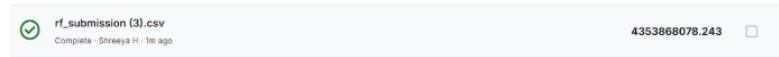


Figure 17: MSE of the predictions on test dataset by Random Forest Regression model

4.8 Gradient Boosting Models

Gradient Boosting is a boosting algorithm where each new model is trained to minimize the mean squared error of the previous model. In each iteration, the algorithm computes the gradient of the loss function with respect to predictions and then trains a new weak model to minimize this gradient. Predictions of the new model are then added to the ensemble (all models' predictions) and the process is repeated until a stopping criterion is met.

4.8.1 XGBoost

XGBoost is a gradient boosting algorithm with extreme regularisation to ensure no overfitting trees are built. The algorithm follows the same approach as gradient boosting but with regularisation methods incorporated into its residual computation and pruning process.

Key Steps:

1. **Computing Splits:** The algorithm computes splits by maximising the gain using a similarity score:

$$\text{Similarity Score} = \frac{(\sum \text{Residuals})^2}{\text{Number of Residuals} + \lambda}$$

where residual = $y_{\text{actual}} - y_{\text{pred}}$

$$\text{Gain} = \text{Similarity}_{\text{Left Child}} + \text{Similarity}_{\text{Right Child}} - \text{Similarity}_{\text{Parent}}$$

2. **Pruning Check:** After finding the best split, the algorithm performs a pruning check using the γ hyperparameter:

Check if $\text{Gain} - \gamma > 0$

- If **YES**: The gain from the split is greater than the complexity cost, so the split is made.
- If **NO**: The gain is not worth the cost of adding a new leaf, so the split is pruned and that branch stops growing.

3. **Making Predictions:** New predictions are computed as:

$$\text{New Prediction} = \text{Previous Prediction} + (\eta \times \text{Output of New Tree})$$

where η is the learning rate.

Hyperparameters:

Hyperparameter	Meaning	Value
n_estimators	Number of trees to be built sequentially	10000
α (reg_alpha)	L1 regularisation term	0.1
λ (reg_lambda)	L2 regularisation term	1
γ (min_split_loos)	Pruning parameter	0.1
η	Learning rate	0.01
min_child_weight	Minimum sum of residuals possible in child node	5
subsample	Percentage of datapoints to be selected randomly	0.8
colsample_by_tree	Percentage of columns to select randomly from dataset	0.8

Table 9: Hyperparameters Description

Model performance: Since XGBoost is a tree-based boosting model it easily captures non-linear patterns from the dataset and the features engineered by us. The built-in regularisation in this model also prevents it from overfitting to the training data.

It gives an RMSE of **42882.643** on the validation dataset and a MSE of **4021822343.407** on the test dataset.

RMSE: 42882.6431

Figure 18: RMSE of the predictions on validation dataset by XGBoost model



Figure 19: MSE of the predictions on test dataset by XGBoost model

4.8.2 CatBoost

CatBoost or Categorical Boosting has two unique features which make it better than XGBoost:

- **Categorical Features Handling:** CatBoost uses a technique called Ordered Target Encoding to encode the categorical data. Firstly it creates randomly shuffled copies of training data (**Random Permutations**) and then iterates through the shuffled data one at a time to compute target encoded value for it using the values which came before it in the shuffled samples. The key difference between this encoding technique and Target Encoding is that it pretends to receive data sequentially one row at a time.
- **Ordered Boosting:** In this type of boosting it randomly shuffles the data and then to get the residual prediction for a data point k , it uses the model trained only on the data points that came before the point k in the random permutation. This way it continues the gradient boosting process while ensuring there is no prediction shift - i.e. it avoids the problem where the model trained on complete data becomes overconfident and passes an artificially small, biased residue to the next model

Hyperparameters:

Hyperparameter	Meaning	Value
iterations	Number of trees to be built sequentially	400
η	Learning rate	0.02
depth	Maximum depth of the decision trees used in boosting process	6
l2_leaf_reg	L2 regularisation parameter (similar to λ in XGBoost)	5
early_stopping_rounds	Model stops training if validation score doesn't improve for this many iterations	100

Table 10: Hyperparameters Description

Model Performance: Due to the ordered boosting algorithm and its ability to handle categorical data well CatBoost regressor performs well on our validation as well as test dataset, effectively catching all the categorical features and non-linear relations.

It gives an RMSE of **43616.881** on the validation dataset and a MSE of **4008143621.562** on the test dataset.

RMSE: 43616.8815

Figure 20: RMSE of the predictions on validation dataset by CatBoost model



Figure 21: MSE of the predictions on test dataset by CatBoost model

4.8.3 LightGBM

LightGBM has optimized techniques such as leaf-wise growth and histogram-based learning make it suitable for handling wide sparse datasets.

Hyperparameters:

Hyperparameter	Meaning	Value
n_estimators	Number of trees to be built sequentially	10000
α (reg_alpha)	L1 regularisation term	0.1
λ (reg_lambda)	L2 regularisation term	0.5
η	Learning rate	0.03
num_leaves	Maximum number of leaves in a tree	32
subsample	Percentage of datapoints to be selected randomly	0.8
colsample_by_tree	Percentage of columns to select randomly from dataset	0.8
max_depth	Maximum depth of a tree	6

Table 11: Hyperparameters Description

Model Performance: Due to our preprocessing efforts in feature engineering and one-hot encoding the low cardinality features and creating a wide sparse dataset, LightGBM is able to apply its specialised optimisation techniques to find the most optimal and impactful splits quickly and optimally.

It gives an RMSE of **42689.398** on the validation dataset and a MSE of **3613576196.541** on the test dataset.

RMSE: 42689.3986

Figure 22: RMSE of the predictions on validation dataset by LightGBM model



Figure 23: MSE of the predictions on test dataset by LightGBM model

4.9 Comparison of all Models

Model	RMSE on Validation Data	MSE on Test Data
Linear Regression	44730.772	6159110982.712
Ridge Regression	44739.314	6057918555.426
LASSO Regression	45210.959	6084636990.011
Elastic Net Regression	45550.297	6097539567.533
Bayesian Regression	45551.300	6122869795.999
KNearest Neighbours	46860.685	6158686669.825
Random Forest Regressor	38316.815	4353868078.243
XGBoost	42882.643	4021822343.407
CatBoost	43616.881	4008143621.562
LightGBM	42689.398	3613576196.541

Table 12: Model Performance Comparison

For the above mentioned models we have finetuned the hyperparameters with the help of GridCVSearch

The LightGBM model has given us the best results in predicting Transport_Cost in the given dataset.

5 References

1. Power Transformer in scikit-learn
<https://www.geeksforgeeks.org/machine-learning/powertransformer-in-scikit-learn/>
2. K Fold Target Encoding
<https://www.geeksforgeeks.org/machine-learning/target-encoding-using-nested-cv-in-sklearn-pipeline/>
3. CatBoost
<https://www.geeksforgeeks.org/machine-learning/catboost-decision-trees-and-boosting-process/>
4. XGBoost
<https://www.geeksforgeeks.org/machine-learning/xgboost/>
5. LightGBM
<https://www.geeksforgeeks.org/machine-learning/lightgbm-light-gradient-boosting-machine/>